

Spring Security

Authentication, authorization and sessions

Today's topics

How Spring checks credentials, remembers a login and restricts access to an API. We will add account registration and connect the security flow to the React frontend.

What you will be able to do

Configure public and protected endpoints, store password hashes, and use browser responses to explain why a request succeeded or failed.

Authentication and authorization

Authentication checks who is making a request. Authorization checks what that identity may do. Spring can allow public reads while requiring an administrator for changes.

Excerpt: `backend/src/main/java/com/example/ecommerce/config/SecurityConfig.java`

```
.requestMatchers(HttpMethod.GET,  
    "/api/products/**", "/api/vendors/**").permitAll()  
.requestMatchers("/api/products/**", "/api/vendors/**")  
    .hasRole("ADMIN")
```

Reading the configuration

`requestMatchers` selects requests by HTTP method and path. `permitAll` allows them without login. `hasRole("ADMIN")` requires an authenticated administrator.

The result

A public product GET can succeed for a guest. A customer cannot change inventory just because the customer has logged in.

Security filters run before the controller

A servlet filter inspects an HTTP request before Spring MVC invokes a controller. Spring Security uses a chain of filters to check credentials, CSRF tokens and access rules.

Excerpt: `backend/src/main/java/com/example/ecommerce/controllers/AuthController.java`

```
@GetMapping("/admin-check")
public String adminCheck() {
    return "Admin access granted";
}
```

This method handles an allowed request

The configuration restricts `/api/auth/admin-check` to ADMIN. Spring returns 401 for a guest or 403 for a customer before this return statement executes.

Where the rule belongs

The controller supplies the response. SecurityConfig supplies the access rule. The method does not manually compare passwords or roles.

Sessions preserve the authenticated identity

HTTP requests arrive separately. After login, Spring stores the authenticated security context in an HTTP session. The browser sends a JSESSIONID cookie that identifies that session on later requests.

Excerpt: `backend/src/main/java/com/example/ecommerce/controllers/AuthController.java`

```
@GetMapping("/me")
public AppUser me(Authentication authentication) {
    return users.findByEmail(authentication.getName())
        .orElseThrow(() ->
            new ResponseStatusException(HttpStatus.UNAUTHORIZED));
}
```

Authentication comes from Spring

The method parameter represents the identity restored from the session. `getName()` is the login email in this application. The caller does not submit a user ID to select an account.

Two kinds of storage

MySQL keeps account records. This example keeps sessions in server memory, so a server restart ends logins without deleting accounts.

Adding Spring Security to the project

Continue with the existing Product/Vendor backend. Add this dependency inside the existing `<dependencies>` element in `pom.xml`, then reload Maven. Keep the current web, JPA and MySQL dependencies.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Database preparation

Run `sql/02_Add_Users.sql` and `sql/03_Add_Cart_Demo.sql` against `ecommerce_day2`. Both preserve existing records. Keep `ddl-auto=validate` after applying them.

Complete the build before connecting React

`Lesson_04_Code.md` contains the six security files and the five original cart files. Include both sets: the storefront requests `/api/cart` immediately after login.

Session settings in application.properties

Add these settings to src/main/resources/application.properties. They control session lifetime and browser cookie behavior. Keep the existing datasource, JPA and logging properties.

Excerpt: backend/src/main/resources/application.properties

```
# Inactivity timeout, not a maximum time since login.  
server.servlet.session.timeout=30m  
server.servlet.session.cookie.http-only=true  
server.servlet.session.cookie.same-site=lax  
# Local HTTP demonstration only. Use secure=true with HTTPS in deployment.  
server.servlet.session.cookie.secure=false
```

timeout and HttpOnly

30m is an inactivity timeout. HttpOnly prevents JavaScript from reading JSESSIONID, while the browser can still send that cookie with requests.

Local and deployed environments

SameSite=Lax restricts some cross-site cookie sending. This local demo uses HTTP. An HTTPS deployment should enable Secure cookies and retain CSRF protection.

The account entity and password privacy

AppUser is the entity for a login account. It stores a display name, a unique email, a password hash and a role. Product and Vendor continue to represent catalog data.

Excerpt: backend/src/main/java/com/example/ecommerce/models/AppUser.java

```
@JsonIgnore
@Column(nullable = false, length = 100)
private String passwordHash;
@Column(nullable = false, length = 20)
private String role;
```

@JsonIgnore

Jackson omits passwordHash when a controller returns AppUser as JSON. Hibernate can still read and write that field in MySQL.

Server-controlled roles

Registration supplies CUSTOMER in Java. A request cannot choose ADMIN or supply an existing entity ID to overwrite another account.

AppUser.java

backend/src/main/java/com/example/ecommerce/models/AppUser.java

```
package com.example.ecommerce.models;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
@Entity
@Table(name = "app_users")
public class AppUser {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, length = 100)
    private String displayName;
    @Column(nullable = false, unique = true, length = 150)
    private String email;
    @JsonIgnore
    @Column(nullable = false, length = 100)
    private String passwordHash;
    @Column(nullable = false, length = 20)
    private String role;
    public AppUser() { }
    public AppUser(String displayName, String email,
        String passwordHash, String role) {
        this.displayName = displayName;
        this.email = email;
        this.passwordHash = passwordHash;
        this.role = role;
    }
    public Long getId() { return id; }
    public String getDisplayName() { return displayName; }
    public String getEmail() { return email; }
    public String getPasswordHash() { return passwordHash; }
    public String getRole() { return role; }
}
```

Complete file. The same source is included in the project and Lesson_04_Code.md.

Account lookup with Spring Data JPA

Spring creates the repository implementation. `findByEmail` loads an account for login. `Optional` represents either a matching account or an absent result. `existsByEmail` supports the registration check.

Complete file: `backend/src/main/java/com/example/ecommerce/repositories/AppUserRepository.java`

```
package com.example.ecommerce.repositories;
import com.example.ecommerce.models.AppUser;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
public interface AppUserRepository extends JpaRepository<AppUser, Long> {
    Optional<AppUser> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

The database still enforces uniqueness

An early `existsByEmail` check improves the response, but the unique email constraint also prevents simultaneous requests from creating duplicates.

Password hashing with BCrypt

A password hash is a one-way representation of a password. BCrypt includes a random salt, so hashing the same password twice normally creates different values. Login checks the submitted password against the stored hash.

Excerpt: `backend/src/main/java/com/example/ecommerce/config/SecurityConfig.java`

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

PasswordEncoder

This interface provides `encode(rawPassword)` for storage and `matches(rawPassword, storedHash)` for verification. Spring's authentication provider performs the login comparison.

The bean

@Bean registers the returned `BCryptPasswordEncoder`. Spring can inject it into `RegistrationService`. Store only the hash and keep passwords and hashes out of responses and logs.

Registration creates an account

Registration accepts a name, email and password. The service validates the fields, checks for a duplicate email, and encodes the password before saving the account.

Excerpt: `backend/src/main/java/com/example/ecommerce/services/RegistrationService.java`

```
String hash = passwordEncoder.encode(password);
AppUser user = new AppUser(name, normalizedEmail, hash, "CUSTOMER");
try {
    AppUser saved = users.save(user);
    log.info("Registered customer id={}", saved.getId());
    return saved;
}
```

The values in this excerpt

password is the submitted text. passwordEncoder is the injected bean. The AppUser constructor receives the hash and the fixed CUSTOMER role.

Expected outcomes

A valid registration returns 201. Invalid fields return 400. A duplicate email returns 409. Registration saves a record but does not log the user in.

RegistrationService.java (1 of 2)

backend/src/main/java/com/example/ecommerce/services/RegistrationService.java

```
package com.example.ecommerce.services;
import com.example.ecommerce.models.AppUser;
import com.example.ecommerce.repositories.AppUserRepository;
import java.nio.charset.StandardCharsets;
import java.util.Locale;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;
@Service
public class RegistrationService {
    private static final Logger log =
        LoggerFactory.getLogger(RegistrationService.class);
    private final AppUserRepository users;
    private final PasswordEncoder passwordEncoder;
    public RegistrationService(AppUserRepository users,
        PasswordEncoder passwordEncoder) {
        this.users = users;
        this.passwordEncoder = passwordEncoder;
    }
    public AppUser register(String displayName, String email, String password) {
        String name = displayName.trim();
        String normalizedEmail = email.trim().toLowerCase(Locale.ROOT);
```

RegistrationService.java (2 of 2)

backend/src/main/java/com/example/ecommerce/services/RegistrationService.java

```
        if (name.isEmpty() || name.length() > 100
            || normalizedEmail.length() > 150
            || !normalizedEmail.matches("[^\\s@]+@[^\\s@]+\\.([^\\s@]+)")
            || password.length() < 8
            || password.getBytes(StandardCharsets.UTF_8).length > 72) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                "Provide a name, email and password of 8 or more characters"
                + " (at most 72 UTF-8 bytes for BCrypt).");
        }
        if (users.existsByEmail(normalizedEmail)) {
            throw new ResponseStatusException(HttpStatus.CONFLICT,
                "Email already registered.");
        }
        String hash = passwordEncoder.encode(password);
        AppUser user = new AppUser(name, normalizedEmail, hash, "CUSTOMER");
        try {
            AppUser saved = users.save(user);
            log.info("Registered customer id={}", saved.getId());
            return saved;
        } catch (DataIntegrityViolationException ex) {
            throw new ResponseStatusException(HttpStatus.CONFLICT,
                "Registration conflicts with an existing record.");
        }
    }
}
```

UserDetailsService connects accounts to login

UserDetailsService is the interface Spring Security calls to load login information. The method name remains loadUserByUsername even when email is the login name. Our implementation uses AppUserRepository.

Excerpt: `backend/src/main/java/com/example/ecommerce/services/DatabaseUserDetailsService.java`

```
public UserDetails loadUserByUsername(String email) {  
    String normalized = email.trim().toLowerCase(Locale.ROOT);  
    AppUser user = users.findByEmail(normalized)  
        .orElseThrow(() ->  
            new UsernameNotFoundException("Invalid credentials"));  
    return User.withUsername(user.getEmail())  
        .password(user.getPasswordHash())  
        .roles(user.getRole())  
        .build();  
}
```

AppUser and UserDetails

AppUser is the database entity. Spring's User implements UserDetails and carries the login name, stored hash and authorities. Building it does not insert another database row.

Who checks the password

This service performs lookup. DaoAuthenticationProvider uses the returned information and PasswordEncoder to verify credentials. Spring assembles that provider from the supplied beans.

DatabaseUserDetailsService.java

backend/src/main/java/com/example/ecommerce/services/DatabaseUserDetailsService.java

```
package com.example.ecommerce.services;
import com.example.ecommerce.models.AppUser;
import com.example.ecommerce.repositories.AppUserRepository;
import java.util.Locale;
import org.springframework.security.core.userdetails.*;
import org.springframework.stereotype.Service;
@Service
public class DatabaseUserDetailsService implements UserDetailsService {
    private final AppUserRepository users;
    public DatabaseUserDetailsService(AppUserRepository users) {
        this.users = users;
    }
    @Override
    public UserDetails loadUserByUsername(String email) {
        String normalized = email.trim().toLowerCase(Locale.ROOT);
        AppUser user = users.findByEmail(normalized)
            .orElseThrow(() ->
                new UsernameNotFoundException("Invalid credentials"));
        return User.withUsername(user.getEmail())
            .password(user.getPasswordHash())
            .roles(user.getRole())
            .build();
    }
}
```

Complete file. The same source is included in the project and Lesson_04_Code.md.

SecurityFilterChain and request rules

HttpSecurity is a builder for security settings. The SecurityFilterChain bean returns the configured chain. Authorization matchers run in order, so a specific public GET rule must precede the broader admin rule.

Excerpt: `backend/src/main/java/com/example/ecommerce/config/SecurityConfig.java`

```
http.authorizeHttpRequests(auth -> auth
    .dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()
    .requestMatchers(HttpMethod.GET, "/api/auth/csrf").permitAll()
    .requestMatchers(HttpMethod.POST,
        "/api/auth/register", "/api/auth/login").permitAll()
    .requestMatchers(HttpMethod.GET,
        "/api/products/**", "/api/vendors/**").permitAll()
    .requestMatchers("/api/products/**", "/api/vendors/**")
        .hasRole("ADMIN")
    .requestMatchers("/api/auth/admin-check").hasRole("ADMIN")
    .requestMatchers("/api/auth/me", "/api/cart/**").authenticated()
    .anyRequest().denyAll());
```

Role names

User.roles("ADMIN") creates the authority ROLE_ADMIN. hasRole("ADMIN") checks that authority. Store ADMIN or CUSTOMER in the database without adding ROLE_.

Unlisted paths and error responses

denyAll rejects unlisted requests. The ERROR dispatcher rule lets Spring render an existing application error without replacing it with another authorization failure.

Login processing and form fields

formLogin installs Spring's username/password authentication mechanism. React submits form-encoded email and password values to the configured URL. Spring's filter processes that request.

Excerpt: `backend/src/main/java/com/example/ecommerce/config/SecurityConfig.java`

```
http.formLogin(form -> form
    .loginProcessingUrl("/api/auth/login")
    .usernameParameter("email")
    .successHandler((request, response, authentication) ->
        response.setStatus(200))
    .failureHandler((request, response, exception) ->
        response.setStatus(401)));
```

Matching the submitted field

`usernameParameter("email")` tells Spring which form field contains the login name. The password field keeps the default name `password`.

Handling the result

Spring supplies request, response and authentication to the success handler. This configuration returns an empty 200 response on success or 401 on bad credentials. React can stay on the current page.

CSRF protection for cookie-based login

A browser sends eligible cookies automatically. A malicious site can try to make it submit an unwanted request while a user is logged in elsewhere. Cross-site request forgery protection adds a token check to modifying requests.

Excerpt: `backend/src/main/java/com/example/ecommerce/controllers/AuthController.java`

```
@GetMapping("/csrf")
public CsrfToken csrf(CsrfToken token) {
    return token;
}
```

Providing the token

Spring resolves `CsrfToken` for this session. The frontend reads its token and headerName properties and attaches that token to POST, PUT and DELETE requests.

Different from the session cookie

JSESSIONID identifies the session and stays `HttpOnly`. The CSRF token is available to JavaScript. A valid token does not grant a role or replace authentication.

The frontend attaches the CSRF header

The API helper fetches `/api/auth/csrf` when it needs a token. It adds the returned header name and token value before sending a modifying request. Components can keep calling the same request function.

Excerpt: `frontend/src/api.js`

```
export async function request(path, method = 'GET', body = null) {
  const headers = {};
  if (!['GET', 'HEAD', 'OPTIONS'].includes(method)) {
    if (!csrf) await refreshCsrf();
    headers[csrf.headerName] = csrf.token;
  }
  const isForm = body instanceof URLSearchParams;
```

Reading the bracket notation

`csrf.headerName` is "X-CSRF-TOKEN" here. `headers[csrf.headerName]` uses that value as the header key. `csrf.token` is the server-generated token value.

Refreshing after login and logout

Spring clears the previous CSRF state at those transitions. `auth.js` fetches a new token after each successful login or logout. A token from the previous session will fail.

Logout ends the server session

Logout must invalidate the authenticated state on the server. Clearing a React variable alone would leave that server session usable. Spring's logout processing performs the session cleanup.

Excerpt: `backend/src/main/java/com/example/ecommerce/config/SecurityConfig.java`

```
http.logout(logout -> logout
    .logoutUrl("/api/auth/logout")
    .invalidateHttpSession(true)
    .deleteCookies("JSESSIONID")
    .logoutSuccessHandler((request, response, authentication) ->
        response.setStatus(204)));
```

What the configuration does

POST `/api/auth/logout` invalidates the session, clears the security context and removes `JSESSIONID`. The successful response is 204 with no body.

CSRF still applies

Logout is a modifying request. The normal frontend logout action sends the current token. A request without it is rejected before logout occurs.

SecurityConfig.java (1 of 2)

backend/src/main/java/com/example/ecommerce/config/SecurityConfig.java

```
package com.example.ecommerce.config;
import jakarta.servlet.DispatcherType;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
@Configuration
public class SecurityConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
        http.csrf(Customizer.withDefaults());
        http.requestCache(cache -> cache.disable());
        http.authorizeHttpRequests(auth -> auth
            .dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()
            .requestMatchers(HttpMethod.GET, "/api/auth/csrf").permitAll()
            .requestMatchers(HttpMethod.POST,
                "/api/auth/register", "/api/auth/login").permitAll()
        );
    }
}
```

One file, part 1 of 2. Copy both parts in order. Full file in Lesson_04_Code.md.

SecurityConfig.java (2 of 2)

backend/src/main/java/com/example/ecommerce/config/SecurityConfig.java

```
        .requestMatchers(HttpMethod.GET,
            "/api/products/**", "/api/vendors/**").permitAll()
        .requestMatchers("/api/products/**", "/api/vendors/**")
            .hasRole("ADMIN")
        .requestMatchers("/api/auth/admin-check").hasRole("ADMIN")
        .requestMatchers("/api/auth/me", "/api/cart/**").authenticated()
        .anyRequest().denyAll());
http.formLogin(form -> form
    .loginProcessingUrl("/api/auth/login")
    .usernameParameter("email")
    .successHandler((request, response, authentication) ->
        response.setStatus(200))
    .failureHandler((request, response, exception) ->
        response.setStatus(401)));
http.logout(logout -> logout
    .logoutUrl("/api/auth/logout")
    .invalidateHttpSession(true)
    .deleteCookies("JSESSIONID")
    .logoutSuccessHandler((request, response, authentication) ->
        response.setStatus(204)));
http.exceptionHandling(errors -> errors
    .authenticationEntryPoint((request, response, exception) ->
        response.setStatus(401))
    .accessDeniedHandler((request, response, exception) ->
        response.setStatus(403)));
return http.build();
    }
}
```

Form fields and the registration controller

@RequestParam reads the named form fields used by this endpoint. The controller passes those strings to RegistrationService. Existing product endpoints continue to accept JSON with @RequestBody.

Excerpt: backend/src/main/java/com/example/ecommerce/controllers/AuthController.java

```
@PostMapping("/register")
@ResponseStatus(HttpStatus.CREATED)
public AppUser register(@RequestParam("displayName") String displayName,
                        @RequestParam("email") String email,
                        @RequestParam("password") String password) {
    return registrationService.register(displayName, email, password);
}
```

How the frontend matches this method

new URLSearchParams({ displayName, email, password }) produces the form body. Those names match the annotation values. Sending a JSON object would require a different controller contract.

The response

@ResponseStatus(CREATED) sets 201 after a successful return. AppUser becomes JSON through Jackson, with passwordHash excluded.

AuthController.java (1 of 2)

backend/src/main/java/com/example/ecommerce/controllers/AuthController.java

```
package com.example.ecommerce.controllers;
import com.example.ecommerce.models.AppUser;
import com.example.ecommerce.repositories.AppUserRepository;
import com.example.ecommerce.services.RegistrationService;
import org.springframework.http.HttpStatus;
import org.springframework.security.core.Authentication;
import org.springframework.security.web.csrf.CsrfToken;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;
@RestController
@RequestMapping("/api/auth")
public class AuthController {
    private final RegistrationService registrationService;
    private final AppUserRepository users;
    public AuthController(RegistrationService registrationService,
        AppUserRepository users) {
        this.registrationService = registrationService;
        this.users = users;
    }
    @GetMapping("/csrf")
    public CsrfToken csrf(CsrfToken token) {
        return token;
    }
}
```

One file, part 1 of 2. Copy both parts in order. Full file in Lesson_04_Code.md.

AuthController.java (2 of 2)

backend/src/main/java/com/example/ecommerce/controllers/AuthController.java

```
@PostMapping("/register")
@ResponseStatus(HttpStatus.CREATED)
public AppUser register(@RequestParam("displayName") String displayName,
    @RequestParam("email") String email,
    @RequestParam("password") String password) {
    return registrationService.register(displayName, email, password);
}
@GetMapping("/me")
public AppUser me(Authentication authentication) {
    return users.findByEmail(authentication.getName())
        .orElseThrow(() ->
            new ResponseStatusException(HttpStatus.UNAUTHORIZED));
}
@GetMapping("/admin-check")
public String adminCheck() {
    return "Admin access granted";
}
}
```

Project structure after the security build

Java files belong under `src/main/java/com/example/ecommerce`. Spring scans beneath the package containing `Application`. The original lesson keeps its cart files together in `shopping`.

Folder	Files and responsibilities
models	<code>AppUser.java</code> stores the login account.
repositories	<code>AppUserRepository.java</code> queries account records.
services	<code>RegistrationService.java</code> and <code>DatabaseUserDetailsService.java</code> create accounts and load login information.
config	<code>SecurityConfig.java</code> supplies security and password-encoder beans.
controllers	<code>AuthController.java</code> supplies registration and account endpoints.
shopping	<code>CartItem</code> , <code>CartItemRepository</code> , <code>CartService</code> , <code>CartController</code> and <code>ShoppingConflictHandler</code> . Full files: code guide , section 5.

Keep existing `Product/Vendor` files in place. Session properties go in `src/main/resources/application.properties`. Keep only one copy of each Java class.

Connecting the supplied React frontend

Run `npm ci` and `npm run dev` inside frontend, then open `http://localhost:5173`. Keep the backend on 8080. Vite forwards relative `/api` requests to the backend, so the browser uses one origin for these calls.

Complete file: `frontend/vite.config.js`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
const proxy = { '/api': 'http://127.0.0.1:8080' };
export default defineConfig({
  plugins: [react()],
  server: { port: 5173, strictPort: true, proxy },
  preview: { port: 4173, strictPort: true, proxy }
});
```

Cookies follow the browser request

`fetch` uses same-origin credentials. The browser stores and sends `JSESSIONID`. React does not copy that cookie into `localStorage` or an `Authorization` header.

Local setup

Use Node.js 22.12 or newer. This proxy is for development. Deployment needs API routing and cookie settings appropriate to the deployed origins.

The frontend login sequence

The login action submits credentials, refreshes the CSRF token, and loads the current account. Each request has a separate job. A successful login response contains no account JSON.

Excerpt: frontend/src/auth.js

```
export async function login(email, password) {  
  await request('/api/auth/login', 'POST',  
    new URLSearchParams({ email, password }));  
  await refreshCsrf();  
  return currentUser();  
}
```

Reading the response

currentUser() requests /api/auth/me to obtain the account for display. The request helper handles JSON, plain text and empty responses instead of calling response.json() unconditionally.

On page refresh

App.jsx calls currentUser() again. A valid session restores the displayed account. A 401 response means this browser is currently signed out.

Observing backend requests in the browser

Open `http://localhost:5173`. Press F12, select Network and filter to Fetch/XHR. Keep the panel open while using the application.

Inspect requests made by the real controls

Register, Log in, Add to cart, the quantity dropdown, Remove and Log out each trigger backend requests. Select a request to inspect its method, URL, status, payload and response.

Confirm the authenticated account

After a successful login, inspect GET `/api/auth/me`. Its 200 response identifies the logged-in account. GET `/api/cart` then returns that account's cart, including `[]` when empty.

Inspect the session cookie

In DevTools, open Application > Cookies for localhost. JSESSIONID should be HttpOnly. A cookie can exist before login because CSRF may create an anonymous session; `/api/auth/me` confirms authentication.

Browser demonstration: registration

Choose Create an account in the frontend. Use a fictional account with these values, then select Register.

Form field	Value
Name	Casey Brooks
Email	casey@example.test
Password	LakeTrail!2026

Inspect the registration response

POST /api/auth/register returns 201 with id, displayName, email and role CUSTOMER. The response must not contain passwordHash.

Account creation and login are separate

The subsequent GET /api/auth/me returns 401 until you log in. If this email already exists, registration returns 409; log in with the existing password instead.

Browser demonstration: login and identity

Use the existing Casey account. Keep Network open to compare the visible interface with the backend responses.

Action	What to observe
Submit the wrong password	POST /api/auth/login returns 401; the interface shows a failure.
Log in with LakeTrail!2026	Login returns 200. React refreshes CSRF, loads /api/auth/me, and loads /api/cart.
Inspect /api/auth/me	200 with Casey's account. This GET uses the session cookie, not the password.
Inspect /api/cart	200 with the cart list. A new empty cart is [], not a 404.
Refresh the browser	The server session restores Casey and the saved cart.

Spring rotates JSESSIONID at login. The stored session identity determines the account. A customer sees shopping controls; administrator inventory controls remain hidden.

Browser demonstration: missing CSRF token

While logged in at <http://localhost:5173>, open DevTools > Console. Run this request directly so it deliberately omits the CSRF header.

```
const response = await fetch('/api/auth/logout', {
  method: 'POST', credentials: 'same-origin'
});
console.log(response.status); // 403
const me = await fetch('/api/auth/me');
console.log(me.status);      // 200
```

Why logout was rejected

The cookie identifies the session, but this modifying request has no CSRF token. Spring rejects it before logout executes. The next GET confirms the account is still logged in.

Compare with the normal Log out button

The header button uses `auth.js` and `api.js`, which attach the CSRF token. That logout returns 204. This deliberate failure is demonstrated in DevTools, not through an extra application button.

Browser demonstration: administrator access

Register Store Admin with email admin@example.test and password RiverStone!2026. Public registration initially assigns CUSTOMER.

Assign the demonstration role

After registration, run the SQL below in MySQL Workbench. Log out and log back in as admin@example.test to establish a session with the new authority.

```
USE ecommerce_day2;  
UPDATE app_users SET role = 'ADMIN'  
WHERE email = 'admin@example.test';
```

Use the administrator interface

The product editor and Edit/Delete controls now appear. Create a product, change its price, and delete that new product while inspecting the Network responses.

A database role change does not update the authorities already stored in an active session. The backend still enforces permissions if someone manually sends a request.

Browser demonstration: logout

Select Log out in the page header. The frontend waits for the server to end the session before showing the signed-out interface.

```
export async function logout() {  
  await request('/api/auth/logout', 'POST');  
  await refreshCsrf();  
}
```

Observe the requests in Network

POST /api/auth/logout returns 204. The helper obtains a fresh CSRF token. The subsequent GET /api/auth/me returns 401 because this browser is now signed out.

Verify the result

Refresh the page. It remains signed out. Log back in: the cart is loaded from MySQL. Ending the HTTP session does not delete the user's account or cart rows.

Cart files used by the frontend

A cart consists of CartItem rows owned by an AppUser. Each row references a Product and stores its quantity. All five original cart files are required for the supplied shopping interface.

File in shopping	Responsibility
CartItem.java	Maps the row, owner, product and quantity.
CartItemRepository.java	Finds cart rows using the logged-in owner's email.
CartService.java	Adds, loads, updates and removes items; calculates demo checkout.
CartController.java	Supplies GET /api/cart and the item POST, PUT and DELETE routes.
ShoppingConflictHandler.java	Returns 409 when a database operation conflicts with related records.

Complete source and SQL: Lesson_04_Code.md, section 5. Adding SQL alone does not add an HTTP endpoint. The frontend's Clear cart button reuses the existing item DELETE method for each line.

Demo checkout takes no payment and stores no order history. PDF receipts are covered later.

Frontend structure

The frontend separates HTTP requests from visual components. Keep each file in its listed location under frontend.

File	Responsibility
src/api.js	Adds CSRF, sends requests, and reads JSON, text or empty responses.
src/auth.js	Registration, login, current-account lookup and logout.
src/App.jsx	Account state, loading data, and event handlers.
src/components/AuthForm.jsx and Header.jsx	Login/registration form and session controls.
src/components/ProductList.jsx and ProductForm.jsx	Catalog, Add to cart, and administrator product editing.
src/components/CartPanel.jsx	Cart quantities, removal, clearing, totals and demo checkout.

index.html and src/main.jsx remain the entry files. src/styles.css supplies the shared layout. All files and imports are included in Lesson_04_Frontend.zip.

Troubleshooting the security flow

Symptom	First check
404 for GET /api/cart after login	Include the five original cart Java files from code-guide section 5. Check that the running backend scans their package. Empty carts return 200 with [].
403 on registration or login	The request must carry the current CSRF token and the session cookie. permitAll does not disable CSRF.
401 after a successful login	Inspect the next /api/auth/me request and its cookie. Keep using localhost:5173 consistently.
403 after an admin role change	Log out and back in. The existing session still has its previous authorities.
Missing cart table at startup	Apply sql/03_Add_Cart_Demo.sql in the configured database before starting with ddl-auto=validate.
Duplicate bean or mapping	Keep one copy of each class. Do not combine the original shopping package with duplicate cart classes from another project.

Use the actual method, path and status in Network to locate the failing request. A successful login and a failed cart load are separate operations.

References

1. Spring Security: servlet architecture

<https://docs.spring.io/spring-security/reference/6.5/servlet/architecture.html>

2. Spring Security: username/password form login

<https://docs.spring.io/spring-security/reference/6.5/servlet/authentication/passwords/form.html>

3. Spring Security: DaoAuthenticationProvider

<https://docs.spring.io/spring-security/reference/6.5/servlet/authentication/passwords/dao-authentication-provider.html>

4. Spring Security: password storage

<https://docs.spring.io/spring-security/reference/6.5/features/authentication/password-storage.html>

5. Spring Security: sessions and authentication persistence

<https://docs.spring.io/spring-security/reference/6.5/servlet/authentication/session-management.html>

6. Spring Security: authorizing HTTP requests

<https://docs.spring.io/spring-security/reference/6.5/servlet/authorization/authorize-http-requests.html>

References

7. Spring Security: CSRF, including the /csrf endpoint

<https://docs.spring.io/spring-security/reference/6.5/servlet/exploits/csrf.html>

8. Spring Security: logout

<https://docs.spring.io/spring-security/reference/6.5/servlet/authentication/logout.html>

9. Spring MVC: request parameters

<https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-methods/requestparam.html>

10. Vite: local API proxy

<https://vite.dev/config/server-options#server-proxy>

11. MDN: using the Fetch API

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

12. Spring Boot: web security auto-configuration

<https://docs.spring.io/spring-boot/3.5/reference/web/spring-security.html>

13. Spring Boot: package structure and component scanning

<https://docs.spring.io/spring-boot/3.5/reference/using/structuring-your-code.html>